CURSO · .NET 8

ARQUITECTURA HEXAGONAL

Y MICROSERVICIOS AVANZADOS

MÓDULO 02

DESARROLLO DE APIs REST CON ASP.NET CORE

PRÁCTICAS

AUTOR: JAIME GALLO

ACERIAAPP · MATERIAL FORMATIVO

M02 — Prácticas de desarrollo de APIs REST

CURSO DE ARQUITECTURA HEXAGONAL Y MICROSERVICIOS AVANZADOS CON .NET 8 AUTOR: JAIME GALLO

Contexto de trabajo

Cada punto reproduce una solución autónoma con src, tests, docs y AceriaApp.sln. El alumno dispone de la solución de referencia y reproduce sus decisiones contrastando cada paso con los archivos entregados. Entorno recomendado: Visual Studio 2026, SDK .NET 8 y SQL Server Express. La base de datos LocalDB puede utilizarse en Windows para escenarios compatibles de desarrollo.

Punto 2.1 — Diseño de APIs REST

Solución: `PROYECTO/2.1-diseno-apis-rest/AceriaApp.sln`. **Origen:** M01/1.5.

PASO 2.1.1 — Preparar copia independiente desde 1.5

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.1-diseno-apis-rest/AceriaApp.sln` y verificar que se está trabajando en el punto 2.1. **Acción 2:** Localizar `AceriaApp.sln` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a M01/1.5. **Acción 3:** Preparar copia independiente desde 1.5. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `AceriaApp.sln` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `AceriaApp.sln` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.1.2 — Abrir solución heredada

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.1-diseno-apis-rest/AceriaApp.sln` y verificar que se está trabajando en el punto 2.1. **Acción 2:** Localizar `AceriaApp.sln` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a M01/1.5. **Acción 3:** Abrir solución heredada. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `AceriaApp.sln` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `AceriaApp.sln` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Consola de comprobación:

```
cd M02_APIS_REST_ASPNET_CORE/PROYECTO/2.1-diseno-apis-rest
dotnet restore AceriaApp.sln
dotnet build AceriaApp.sln
```

PASO 2.1.3 — Verificar documentos heredados

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.1-diseno-apis-rest/AceriaApp.sln` y verificar que se está trabajando en el punto 2.1. **Acción 2:** Localizar `docs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a M01/1.5. **Acción 3:** Verificar documentos heredados. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `docs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `docs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.1.4 — Definir contrato de diseño REST

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.1-diseno-apis-rest/AceriaApp.sln` y verificar que se está trabajando en el punto 2.1. **Acción 2:** Localizar `docs/diseno-apis-rest.md` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a M01/1.5. **Acción 3:** Definir contrato de diseño REST. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `docs/diseno-apis-rest.md` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `docs/diseno-apis-rest.md` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.1.5 — Crear carpeta DisenoRest

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.1-diseno-apis-rest/AceriaApp.sln` y verificar que se está trabajando en el punto 2.1. **Acción 2:** Localizar `src/AceriaApp.Monolito.Api/DisenoRest` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a M01/1.5. **Acción 3:** Crear carpeta DisenoRest. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `src/AceriaApp.Monolito.Api/DisenoRest` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `src/AceriaApp.Monolito.Api/DisenoRest` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.1.6 — Crear clase UriRest

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.1-diseno-apis-rest/AceriaApp.sln` y verificar que se está trabajando en el punto 2.1. **Acción 2:** Localizar `src/AceriaApp.Monolito.Api/DisenoRest/UriRest.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a

M01/1.5. **Acción 3:** Crear clase UriRest. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `src/AceriaApp.Monolito.Api/DisenoRest/UriRest.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `src/AceriaApp.Monolito.Api/DisenoRest/UriRest.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Código real de referencia:

```
namespace AceriaApp.Monolito.Api.DisenoRest;
public class UriRest
{
    public string Version { get; init; } = "v1";
    public string Recurso { get; init; } = string.Empty;
    public string? Identificador { get; init; }
    public string? SubRecurso { get; init; }
    public string Construir()
    {
        var partes = new List<string> { "api", Version, Recurso };
        if (!string.IsNullOrWhiteSpace(Identificador)) partes.Add(Identificador);
        if (!string.IsNullOrWhiteSpace(SubRecurso)) partes.Add(SubRecurso);
        return "/" + string.Join("/", partes);
    }
}
```

El alumno identifica tipo, parámetros, valor devuelto y caso de error en el archivo anterior.

PASO 2.1.7 — Revisar construcción de URIs

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.1-diseno-apis-rest/AceriaApp.sln` y verificar que se está trabajando en el punto 2.1. **Acción 2:** Localizar `src/AceriaApp.Monolito.Api/DisenoRest/UriRest.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a M01/1.5. **Acción 3:** Revisar construcción de URIs. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `src/AceriaApp.Monolito.Api/DisenoRest/UriRest.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `src/AceriaApp.Monolito.Api/DisenoRest/UriRest.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Código real de referencia:

```
namespace AceriaApp.Monolito.Api.DisenoRest;
public class UriRest
{
    public string Version { get; init; } = "v1";
    public string Recurso { get; init; } = string.Empty;
    public string? Identificador { get; init; }
    public string? SubRecurso { get; init; }
    public string Construir()
    {
        var partes = new List<string> { "api", Version, Recurso };
        if (!string.IsNullOrWhiteSpace(Identificador)) partes.Add(Identificador);
        if (!string.IsNullOrWhiteSpace(SubRecurso)) partes.Add(SubRecurso);
        return "/" + string.Join("/", partes);
    }
}
```

El alumno identifica tipo, parámetros, valor devuelto y caso de error en el archivo anterior.

PASO 2.1.8 — Crear ValidadorUriRest

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.1-diseno-apis-rest/AceriaApp.sln` y verificar que se está trabajando en el punto 2.1. **Acción 2:** Localizar `src/AceriaApp.Monolito.Api/DisenoRest/ValidadorUriRest.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a M01/1.5. **Acción 3:** Crear ValidadorUriRest. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `src/AceriaApp.Monolito.Api/DisenoRest/ValidadorUriRest.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `src/AceriaApp.Monolito.Api/DisenoRest/ValidadorUriRest.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Código real de referencia:

```
using System.Text.RegularExpressions;
namespace AceriaApp.Monolito.Api.DisenoRest;
public class ValidadorUriRest
{
    private static readonly HashSet<string> VerbosProhibidos = new(StringComparer.OrdinalIgnoreCase)
        { "crear", "obtener", "listar", "actualizar", "eliminar", "get", "post", "put", "patch",
"delete" };
    public ResultadoUri Validar(string? uri)
    {
        var errores = new List<string>();
        if (string.IsNullOrWhiteSpace(uri)) return new(false, new[] { "La URI es obligatoria." });
        if (!uri.StartsWith("/api/", StringComparison.Ordinal)) errores.Add("La URI no empieza por
'/api/'.");
        var partes = uri.Split('?', 2)[0].Trim('/').Split('/');
        if (partes.Length < 3 || !Regex.IsMatch(partes[1], @"^v[1-9]\d*$"))
            errores.Add("La URI no incluye versión después de 'api'.");
        if (partes.Length < 3 || string.IsNullOrWhiteSpace(partes[2]))
            errores.Add("La URI no identifica un recurso.");
        foreach (var parte in partes.Skip(2))
            if (VerbosProhibidos.Contains(parte))
                errores.Add($"La URI contiene un verbo prohibido: '{parte}'.");
        return new(errores.Count==0,errores);
    }
}
public record ResultadoUri(bool EsValida, IReadOnlyList<string> Errores);
```

El alumno identifica tipo, parámetros, valor devuelto y caso de error en el archivo anterior.

PASO 2.1.9 — Probar versión y segmentos de acción

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.1-diseno-apis-rest/AceriaApp.sln` y verificar que se está trabajando en el punto 2.1. **Acción 2:** Localizar `src/AceriaApp.Monolito.Api/DisenoRest/ValidadorUriRest.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a M01/1.5. **Acción 3:** Probar versión y segmentos de acción. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `src/AceriaApp.Monolito.Api/DisenoRest/ValidadorUriRest.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `src/AceriaApp.Monolito.Api/DisenoRest/ValidadorUriRest.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Código real de referencia:

```
using System.Text.RegularExpressions;
namespace AceriaApp.Monolito.Api.DisenoRest;
public class ValidadorUriRest
{
    private static readonly HashSet<string> VerbosProhibidos = new(StringComparer.OrdinalIgnoreCase)
        { "crear", "obtener", "listar", "actualizar", "eliminar", "get", "post", "put", "patch",
"delete" };
    public ResultadoUri Validar(string? uri)
    {
        var errores = new List<string>();
        if (string.IsNullOrWhiteSpace(uri)) return new(false, new[] { "La URI es obligatoria." });
        if (!uri.StartsWith("/api/", StringComparison.Ordinal)) errores.Add("La URI no empieza por
'/api/'.");
        var partes = uri.Split('?', 2)[0].Trim('/').Split('/');
        if (partes.Length < 3 || !Regex.IsMatch(partes[1], @"^v[1-9]\d*$"))
            errores.Add("La URI no incluye versión después de 'api'.");
        if (partes.Length < 3 || string.IsNullOrWhiteSpace(partes[2]))
            errores.Add("La URI no identifica un recurso.");
        foreach (var parte in partes.Skip(2))
            if (VerbosProhibidos.Contains(parte))
                errores.Add($"La URI contiene un verbo prohibido: '{parte}'.");
        return new(errores.Count==0,errores);
    }
}
public record ResultadoUri(bool EsValida, IReadOnlyList<string> Errores);
```

El alumno identifica tipo, parámetros, valor devuelto y caso de error en el archivo anterior.

PASO 2.1.10 — Crear CatalogoUris

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.1-diseno-apis-rest/AceriaApp.sln` y verificar que se está trabajando en el punto 2.1. **Acción 2:** Localizar `src/AceriaApp.Monolito.Api/DisenoRest/CatalogoUris.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a M01/1.5. **Acción 3:** Crear CatalogoUris. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `src/AceriaApp.Monolito.Api/DisenoRest/CatalogoUris.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `src/AceriaApp.Monolito.Api/DisenoRest/CatalogoUris.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.1.11 — Comprobar consultas de catálogo

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.1-diseno-apis-rest/AceriaApp.sln` y verificar que se está trabajando en el punto 2.1. **Acción 2:** Localizar `src/AceriaApp.Monolito.Api/DisenoRest/CatalogoUris.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a M01/1.5. **Acción 3:** Comprobar consultas de catálogo. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `src/AceriaApp.Monolito.Api/DisenoRest/CatalogoUris.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `src/AceriaApp.Monolito.Api/DisenoRest/CatalogoUris.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.1.12 — Añadir proyecto de pruebas

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.1-diseno-apis-rest/AceriaApp.sln` y verificar que se está trabajando en el punto 2.1. **Acción 2:** Localizar `tests/AceriaApp.DisenoRest.Tests` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a M01/1.5. **Acción 3:** Añadir proyecto de pruebas. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet test AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `tests/AceriaApp.DisenoRest.Tests` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `tests/AceriaApp.DisenoRest.Tests` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.1.13 — Añadir ProjectReference

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.1-diseno-apis-rest/AceriaApp.sln` y verificar que se está trabajando en el punto 2.1. **Acción 2:** Localizar `tests/AceriaApp.DisenoRest.Tests/AceriaApp.DisenoRest.Tests.csproj` y comprobar su función dentro del proyecto acumulativo. No modificar

la solución correspondiente a M01/1.5. **Acción 3:** Añadir ProjectReference. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `tests/AceriaApp.DisenoRest.Tests/AceriaApp.DisenoRest.Tests.csproj` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `tests/AceriaApp.DisenoRest.Tests/AceriaApp.DisenoRest.Tests.csproj` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.1.14 — Revisar paquetes de pruebas

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.1-diseno-apis-rest/AceriaApp.sln` y verificar que se está trabajando en el punto 2.1. **Acción 2:** Localizar `tests/AceriaApp.DisenoRest.Tests/AceriaApp.DisenoRest.Tests.csproj` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a M01/1.5. **Acción 3:** Revisar paquetes de pruebas. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet test AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `tests/AceriaApp.DisenoRest.Tests/AceriaApp.DisenoRest.Tests.csproj` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `tests/AceriaApp.DisenoRest.Tests/AceriaApp.DisenoRest.Tests.csproj` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.1.15 — Testear URIs válidas e inválidas

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.1-diseno-apis-rest/AceriaApp.sln` y verificar que se está trabajando en el punto 2.1. **Acción 2:** Localizar `tests/AceriaApp.DisenoRest.Tests/ValidadorUriRestTests.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a M01/1.5. **Acción 3:** Testear URIs válidas e inválidas. Revisar el contenido

completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet test AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `tests/AceriaApp.DisenoRest.Tests/ValidadorUriRestTests.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `tests/AceriaApp.DisenoRest.Tests/ValidadorUriRestTests.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Código real de referencia:

```
using AceriaApp.Monolito.Api.DisenoRest;
using Xunit;
namespace AceriaApp.DisenoRest.Tests;
public class ValidadorUriRestTests
{
    private readonly ValidadorUriRest _v = new();
    [Fact] public void Valida_UriDeColeccion() { Assert.True(_v.Validar("/api/v1/
ordenes").EsValida); }
    [Fact] public void Rechaza_SinPrefijo() { Assert.False(_v.Validar("/ordenes").EsValida); }
    [Fact] public void Rechaza_SinVersion() { Assert.False(_v.Validar("/api/ordenes").EsValida); }
    [Fact] public void Rechaza_SegmentoDeAccion() { Assert.False(_v.Validar("/api/v1/
crear").EsValida); }
}
```

El alumno identifica tipo, parámetros, valor devuelto y caso de error en el archivo anterior.

PASO 2.1.16 — Testear construcción de rutas

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.1-diseno-apis-rest/AceriaApp.sln` y verificar que se está trabajando en el punto 2.1. **Acción 2:** Localizar `tests/AceriaApp.DisenoRest.Tests/UriRestTests.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a M01/1.5. **Acción 3:** Testear construcción de rutas. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet test AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `tests/AceriaApp.DisenoRest.Tests/UriRestTests.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.

- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `tests/AceriaApp.DisenoRest.Tests/UriRestTests.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Código real de referencia:

```
using AceriaApp.Monolito.Api.DisenoRest;
using Xunit;
namespace AceriaApp.DisenoRest.Tests;
public class UriRestTests
{
    [Fact] public void Construir_Coleccion() => Assert.Equal("/api/v1/ordenes",new UriRest
{Recurso="ordenes"}.Construir());
    [Fact] public void Construir_Identificador() => Assert.Equal("/api/v1/ordenes/123",new UriRest
{Recurso="ordenes", Identificador="123"}.Construir());
    [Fact] public void Construir_SubRecurso() => Assert.Equal("/api/v1/ordenes/123/lineas",new
UriRest {Recurso="ordenes", Identificador="123", SubRecurso="lineas"}.Construir());
}
```

El alumno identifica tipo, parámetros, valor devuelto y caso de error en el archivo anterior.

PASO 2.1.17 — Testear partición del catálogo

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.1-diseno-apis-rest/AceriaApp.sln` y verificar que se está trabajando en el punto 2.1. **Acción 2:** Localizar `tests/AceriaApp.DisenoRest.Tests/CatalogoUrisTests.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a M01/1.5. **Acción 3:** Testear partición del catálogo. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet test AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `tests/AceriaApp.DisenoRest.Tests/CatalogoUrisTests.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `tests/AceriaApp.DisenoRest.Tests/CatalogoUrisTests.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.1.18 — Compilar solución

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.1-diseno-apis-rest/AceriaApp.sln` y verificar que se está trabajando en el punto 2.1. **Acción 2:** Localizar `AceriaApp.sln` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a M01/1.5. **Acción 3:** Compilar solución.

Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `AceriaApp.sln` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `AceriaApp.sln` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.1.19 — Ejecutar ocho tests nuevos

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.1-diseno-apis-rest/AceriaApp.sln` y verificar que se está trabajando en el punto 2.1. **Acción 2:** Localizar `tests/AceriaApp.DisenoRest.Tests` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a M01/1.5. **Acción 3:** Ejecutar ocho tests nuevos. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet test AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `tests/AceriaApp.DisenoRest.Tests` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `tests/AceriaApp.DisenoRest.Tests` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.1.20 — Ejecutar tests acumulados

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.1-diseno-apis-rest/AceriaApp.sln` y verificar que se está trabajando en el punto 2.1. **Acción 2:** Localizar `AceriaApp.sln` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a M01/1.5. **Acción 3:** Ejecutar tests acumulados. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet test AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `AceriaApp.sln` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `AceriaApp.sln` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.1.21 — Actualizar README del punto

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.1-diseno-apis-rest/AceriaApp.sln` y verificar que se está trabajando en el punto 2.1. **Acción 2:** Localizar `README.md` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a M01/1.5. **Acción 3:** Actualizar README del punto. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `README.md` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `README.md` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.1.22 — Crear ADR-006

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.1-diseno-apis-rest/AceriaApp.sln` y verificar que se está trabajando en el punto 2.1. **Acción 2:** Localizar `docs/ADR-006-diseno-apis-rest.md` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a M01/1.5. **Acción 3:** Crear ADR-006. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `docs/ADR-006-diseno-apis-rest.md` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `docs/ADR-006-diseno-apis-rest.md` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.1.23 — Comparar documentos anteriores

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.1-diseno-apis-rest/AceriaApp.sln` y verificar que se está trabajando en el punto 2.1. **Acción 2:** Localizar `docs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a M01/1.5. **Acción 3:** Comparar documentos anteriores. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `docs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `docs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Verificación íntegra en CMD desde la raíz del repositorio:

```
fc /L "M01_INTRODUCCION_MICROSERVICIOS\PROYECTO\1.5-buenas-practicas\docs\ADR-005-buenas-practicas.md" "M02_APIS_REST_ASPNET_CORE\PROYECTO\2.1-diseno-apis-rest\docs\ADR-005-buenas-practicas.md"
```

Para un archivo .bat debe utilizarse %%f; desde CMD interactivo se usa %f.

PASO 2.1.24 — Confirmar el estado final

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.1-diseno-apis-rest/AceriaApp.sln` y verificar que se está trabajando en el punto 2.1. **Acción 2:** Localizar `AceriaApp.sln` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a M01/1.5. **Acción 3:** Confirmar el estado final. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `AceriaApp.sln` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `AceriaApp.sln` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Tabla de errores del ejercicio completo

Error	Causa	Solución
No se encuentra la solución	Directorio incorrecto	Cambiar a la carpeta del punto y abrir AceriaApp.sln.
No se resuelve un tipo	Proyecto o dependencia ausente	Comprobar csproj y ejecutar dotnet restore.
No se publica Swagger	Entorno o middleware incorrecto	Contrastar Development con Production y consultar el JSON real.
Fallan tests anteriores	Cambio en el estado heredado	Restaurar el archivo alterado desde la carpeta anterior.
La documentación difiere del código	Contrato no actualizado	Comprobar ruta, modelo y respuesta con la API ejecutada.

Reto resuelto paso a paso

Reto: ampliar CatalogoUris con un resumen de válidas, inválidas y porcentaje.

Paso 1: Crear un record con Validas, Invalidas y PorcentajeValidez. **Paso 2:** Agregar el método Resumen dentro de la clase indicada. **Paso 3:** Contar las entradas válidas a través del validador. **Paso 4:** Calcular las inválidas como total menos válidas. **Paso 5:** Proteger el denominador cuando el total sea cero. **Paso 6:** Incorporar un test con tres válidas y dos inválidas. **Paso 7:** Ejecutar el proyecto de pruebas y comprobar 60%. **Paso 8:** Conservar la solución previa sin alteraciones.

Referencia de cálculo: `_uris.Count(u => validador.Validar(u).EsValida)`. Esta ampliación está indicada en el código como reto, pero no forma parte del recuento base hasta que se implemente.

Resultado esperado

- Solución independiente con 7 proyectos; una API y 6 proyectos de tests.
- Recuento acumulado previsto de pruebas: 31.
- Todos los documentos y proyectos de los puntos anteriores permanecen accesibles.

Conclusión y enlace tras el punto 2.1

El punto 2.1 vincula diseño de apis rest con clases y comprobaciones concretas. El alumnado puede inspeccionar el proyecto y explicar los resultados observados antes de abordar el siguiente punto.

Punto 2.2 — Recursos, verbos HTTP y códigos de respuesta

Solución: `PROYECTO/2.2-recursos-verbos-codigos/AceriaApp.sln`. **Origen:** 2.1-diseno-apis-rest.

PASO 2.2.1 — Copiar solución del punto 2.1

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.2-recursos-verbos-codigos/AceriaApp.sln` y verificar que se está trabajando en el punto 2.2. **Acción 2:** Localizar `AceriaApp.sln` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.1-diseno-apis-rest. **Acción 3:** Copiar solución del punto 2.1. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `AceriaApp.sln` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `AceriaApp.sln` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.2.2 — Abrir solución 2.2

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.2-recursos-verbos-codigos/AceriaApp.sln` y verificar que se está trabajando en el punto 2.2. **Acción 2:** Localizar `AceriaApp.sln` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.1-diseno-apis-rest. **Acción 3:** Abrir solución 2.2. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `AceriaApp.sln` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `AceriaApp.sln` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Consola de comprobación:

```
cd M02_APIS_REST_ASPNET_CORE/PROYECTO/2.2-recursos-verbos-codigos
dotnet restore AceriaApp.sln
dotnet build AceriaApp.sln
```

PASO 2.2.3 — Revisar documentación recibida

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.2-recursos-verbos-codigos/AceriaApp.sln` y verificar que se está trabajando en el punto 2.2. **Acción 2:** Localizar `docs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.1-diseno-apis-rest. **Acción 3:** Revisar documentación recibida. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `docs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `docs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.2.4 — Escribir catálogo de recursos

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.2-recursos-verbos-codigos/AceriaApp.sln` y verificar que se está trabajando en el punto 2.2. **Acción 2:** Localizar `docs/catalogo-recursos.md` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.1-diseno-apis-rest. **Acción 3:** Escribir catálogo de recursos. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `docs/catalogo-recursos.md` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `docs/catalogo-recursos.md` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.2.5 — Crear carpeta CatalogoRecursos

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.2-recursos-verbos-codigos/AceriaApp.sln` y verificar que se está trabajando en el punto 2.2. **Acción 2:** Localizar `src/AceriaApp.Monolito.Api/CatalogoRecursos` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.1-diseno-apis-rest. **Acción 3:** Crear carpeta CatalogoRecursos. Revisar el contenido completo del

elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `src/AceriaApp.Monolito.Api/CatalogoRecursos` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `src/AceriaApp.Monolito.Api/CatalogoRecursos` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.2.6 — Crear CombinacionVerboCodigo

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.2-recursos-verbos-codigos/AceriaApp.sln` y verificar que se está trabajando en el punto 2.2. **Acción 2:** Localizar `src/AceriaApp.Monolito.Api/CatalogoRecursos/CombinacionVerboCodigo.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.1-diseno-apis-rest. **Acción 3:** Crear CombinacionVerboCodigo. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `src/AceriaApp.Monolito.Api/CatalogoRecursos/CombinacionVerboCodigo.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `src/AceriaApp.Monolito.Api/CatalogoRecursos/CombinacionVerboCodigo.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Código real de referencia:

```
namespace AceriaApp.Monolito.Api.CatalogoRecursos;
public class CombinacionVerboCodigo
{
    public string Verbo { get; init; }=string.Empty;
    public int Codigo { get; init; }
}
```

El alumno identifica tipo, parámetros, valor devuelto y caso de error en el archivo anterior.

PASO 2.2.7 — Crear ValidadorCombinacionVerboCodigo

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.2-recursos-verbos-codigos/AceriaApp.sln` y verificar que se está trabajando en el punto 2.2. **Acción 2:** Localizar `src/AceriaApp.Monolito.Api/CatalogoRecursos/ValidadorCombinacionVerboCodigo.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.1-diseno-apis-rest. **Acción 3:** Crear ValidadorCombinacionVerboCodigo. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `src/AceriaApp.Monolito.Api/CatalogoRecursos/ValidadorCombinacionVerboCodigo.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `src/AceriaApp.Monolito.Api/CatalogoRecursos/ValidadorCombinacionVerboCodigo.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Código real de referencia:

```
namespace AceriaApp.Monolito.Api.CatalogoRecursos;
public class ValidadorCombinacionVerboCodigo
{
    private static readonly Dictionary<string,int[]>
CodigosPorVerbo=new(StringComparer.OrdinalIgnoreCase)
    {
        ["GET"]=new[]{200,404}, ["POST"]=new[]{201,400,409,422},
        ["PUT"]=new[]{200,204,400,404,409,422},
        ["PATCH"]=new[]{200,204,400,404,409,422},
        ["DELETE"]=new[]{204,404}
    };
    public ResultadoCombinacion Validar(CombinacionVerboCodigo combinacion)
    {
        var errores=new List<string>();
        if(!CodigosPorVerbo.TryGetValue(combinacion.Verbo,out var codigos))
        {
            errores.Add($"Verbo no permitido: '{combinacion.Verbo}'.");
            return new(false,errores);
        }
        if(!codigos.Contains(combinacion.Codigo))
            errores.Add($"Código '{combinacion.Codigo}' no permitido para el verbo
'{combinacion.Verbo}'.");
        return new(errores.Count==0,errores);
    }
}
public record ResultadoCombinacion(bool EsValida,IReadOnlyList<string> Errores);
```

El alumno identifica tipo, parámetros, valor devuelto y caso de error en el archivo anterior.

PASO 2.2.8 — Inspeccionar matriz de estados

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.2-recursos-verbos-codigos/AceriaApp.sln` y verificar que se está trabajando en el punto 2.2. **Acción 2:** Localizar `src/AceriaApp.Monolito.Api/CatalogoRecursos/ValidadorCombinacionVerboCodigo.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.1-diseno-apis-rest. **Acción 3:** Inspeccionar matriz de estados. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `src/AceriaApp.Monolito.Api/CatalogoRecursos/ValidadorCombinacionVerboCodigo.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `src/AceriaApp.Monolito.Api/CatalogoRecursos/ValidadorCombinacionVerboCodigo.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Código real de referencia:

```
namespace AceriaApp.Monolito.Api.CatalogoRecursos;
public class ValidadorCombinacionVerboCodigo
{
    private static readonly Dictionary<string,int[]>
CodigosPorVerbo=new(StringComparer.OrdinalIgnoreCase)
    {
        ["GET"]=new[]{200,404}, ["POST"]=new[]{201,400,409,422},
        ["PUT"]=new[]{200,204,400,404,409,422},
        ["PATCH"]=new[]{200,204,400,404,409,422},
        ["DELETE"]=new[]{204,404}
    };
    public ResultadoCombinacion Validar(CombinacionVerboCodigo combinacion)
    {
        var errores=new List<string>();
        if(!CodigosPorVerbo.TryGetValue(combinacion.Verbo,out var codigos))
        {
            errores.Add($"Verbo no permitido: '{combinacion.Verbo}'.");
            return new(false,errores);
        }
        if(!codigos.Contains(combinacion.Codigo))
            errores.Add($"Código '{combinacion.Codigo}' no permitido para el verbo
'{combinacion.Verbo}'.");
        return new(errores.Count==0,errores);
    }
}
public record ResultadoCombinacion(bool EsValida,IReadOnlyList<string> Errores);
```

El alumno identifica tipo, parámetros, valor devuelto y caso de error en el archivo anterior.

PASO 2.2.9 — Crear CatalogoCombinaciones

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.2-recursos-verbos-codigos/AceriaApp.sln` y verificar que se está trabajando en el punto 2.2. **Acción 2:** Localizar `src/AceriaApp.Monolito.Api/CatalogoRecursos/CatalogoCombinaciones.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.1-diseno-apis-rest. **Acción 3:** Crear CatalogoCombinaciones. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `src/AceriaApp.Monolito.Api/CatalogoRecursos/CatalogoCombinaciones.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `src/AceriaApp.Monolito.Api/CatalogoRecursos/CatalogoCombinaciones.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Código real de referencia:

```
namespace AceriaApp.Monolito.Api.CatalogoRecursos;
public class CatalogoCombinaciones
{
    private readonly IReadOnlyList<CombinacionVerboCodigo> _combinaciones;
    public CatalogoCombinaciones(IEnumerable<CombinacionVerboCodigo> combinaciones) =>
        _combinaciones=combinaciones.ToArray();
    public IEnumerable<CombinacionVerboCodigo> CombinacionesValidas(ValidadorCombinacionVerboCodigo
v) =>
        _combinaciones.Where(c=>v.Validar(c).EsValida);
    public IEnumerable<CombinacionVerboCodigo>
CombinacionesInvalidas(ValidadorCombinacionVerboCodigo v) =>
        _combinaciones.Where(c=>!v.Validar(c).EsValida);
    // RETO 2.2: agregar Resumen y record ResumenCombinaciones.
}
```

El alumno identifica tipo, parámetros, valor devuelto y caso de error en el archivo anterior.

PASO 2.2.10 — Comprobar filtrado con LINQ

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.2-recursos-verbos-codigos/AceriaApp.sln` y verificar que se está trabajando en el punto 2.2. **Acción 2:** Localizar `src/AceriaApp.Monolito.Api/CatalogoRecursos/CatalogoCombinaciones.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.1-diseno-apis-rest. **Acción 3:** Comprobar filtrado con LINQ. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `src/AceriaApp.Monolito.Api/CatalogoRecursos/CatalogoCombinaciones.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `src/AceriaApp.Monolito.Api/CatalogoRecursos/CatalogoCombinaciones.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.2.11 — Añadir proyecto de pruebas

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.2-recursos-verbos-codigos/AceriaApp.sln` y verificar que se está trabajando en el punto 2.2. **Acción 2:** Localizar `tests/AceriaApp.CatalogoRecursos.Tests` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.1-diseno-apis-rest. **Acción 3:** Añadir proyecto de pruebas. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet test AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `tests/AceriaApp.CatalogoRecursos.Tests` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `tests/AceriaApp.CatalogoRecursos.Tests` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.2.12 — Añadir referencia al monolito

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.2-recursos-verbos-codigos/AceriaApp.sln` y verificar que se está trabajando en el punto 2.2. **Acción 2:** Localizar `tests/AceriaApp.CatalogoRecursos.Tests/AceriaApp.CatalogoRecursos.Tests.csproj` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.1-diseno-apis-rest. **Acción 3:** Añadir referencia al monolito. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `tests/AceriaApp.CatalogoRecursos.Tests/AceriaApp.CatalogoRecursos.Tests.csproj` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `tests/AceriaApp.CatalogoRecursos.Tests/AceriaApp.CatalogoRecursos.Tests.csproj` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.2.13 — Revisar dependencias xUnit

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.2-recursos-verbos-codigos/AceriaApp.sln` y verificar que se está trabajando en el punto 2.2. **Acción 2:** Localizar `tests/AceriaApp.CatalogoRecursos.Tests/AceriaApp.CatalogoRecursos.Tests.csproj` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.1-diseno-apis-rest. **Acción 3:** Revisar dependencias xUnit. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `tests/AceriaApp.CatalogoRecursos.Tests/AceriaApp.CatalogoRecursos.Tests.csproj` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `tests/AceriaApp.CatalogoRecursos.Tests/AceriaApp.CatalogoRecursos.Tests.csproj` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.2.14 — Probar casos de la matriz

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.2-recursos-verbos-codigos/AceriaApp.sln` y verificar que se está trabajando en el punto 2.2. **Acción 2:** Localizar `tests/AceriaApp.CatalogoRecursos.Tests/ValidadorCombinacionVerboCodigoTests.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.1-diseno-apis-rest. **Acción 3:** Probar casos de la matriz. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `tests/AceriaApp.CatalogoRecursos.Tests/ValidadorCombinacionVerboCodigoTests.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `tests/AceriaApp.CatalogoRecursos.Tests/ValidadorCombinacionVerboCodigoTests.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.2.15 — Probar catálogo

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.2-recursos-verbos-codigos/AceriaApp.sln` y verificar que se está trabajando en el punto 2.2. **Acción 2:** Localizar `tests/AceriaApp.CatalogoRecursos.Tests/CatalogoCombinacionesTests.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.1-diseno-apis-rest. **Acción 3:** Probar catálogo. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `tests/AceriaApp.CatalogoRecursos.Tests/CatalogoCombinacionesTests.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `tests/AceriaApp.CatalogoRecursos.Tests/CatalogoCombinacionesTests.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Código real de referencia:

```
using AceriaApp.Monolito.Api.CatalogoRecursos;
using Xunit;
namespace AceriaApp.CatalogoRecursos.Tests;
public class CatalogoCombinacionesTests
{
    [Fact] public void Catalogo_SeparaConjuntos()
    {
        var c=new CatalogoCombinaciones(new CombinacionVerboCodigo[]{
            new(){Verbo="GET",Codigo=200},new(){Verbo="POST",Codigo=201},
            new(){Verbo="GET",Codigo=201},new(){Verbo="OPTIONS",Codigo=200}});
        var v=new ValidadorCombinacionVerboCodigo();
        Assert.Equal(2,c.CombinacionesValidas(v).Count());
        Assert.Equal(2,c.CombinacionesInvalidas(v).Count());
    }
}
```

El alumno identifica tipo, parámetros, valor devuelto y caso de error en el archivo anterior.

PASO 2.2.16 — Compilar solución

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.2-recursos-verbos-codigos/AceriaApp.sln` y verificar que se está trabajando en el punto 2.2. **Acción 2:** Localizar `AceriaApp.sln` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.1-diseno-apis-rest. **Acción 3:** Compilar solución. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `AceriaApp.sln` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `AceriaApp.sln` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.2.17 — Ejecutar cinco tests nuevos

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.2-recursos-verbos-codigos/AceriaApp.sln` y verificar que se está trabajando en el punto 2.2. **Acción 2:** Localizar `tests/AceriaApp.CatalogoRecursos.Tests` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.1-diseno-apis-rest. **Acción 3:** Ejecutar cinco tests nuevos. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet test AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `tests/AceriaApp.CatalogoRecursos.Tests` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `tests/AceriaApp.CatalogoRecursos.Tests` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.2.18 — Ejecutar tests de la solución

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.2-recursos-verbos-codigos/AceriaApp.sln` y verificar que se está trabajando en el punto 2.2. **Acción 2:** Localizar `AceriaApp.sln` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.1-diseno-apis-rest. **Acción 3:** Ejecutar tests de la solución. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet test AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `AceriaApp.sln` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `AceriaApp.sln` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.2.19 — Actualizar README

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.2-recursos-verbos-codigos/AceriaApp.sln` y verificar que se está trabajando en el punto 2.2. **Acción 2:** Localizar `README.md` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.1-diseno-apis-rest. **Acción 3:** Actualizar README. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `README.md` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `README.md` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.2.20 — Crear ADR-007

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.2-recursos-verbos-codigos/AceriaApp.sln` y verificar que se está trabajando en el punto 2.2. **Acción 2:** Localizar `docs/ADR-007-catalogo-recursos.md` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.1-diseno-apis-rest. **Acción 3:** Crear ADR-007. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `docs/ADR-007-catalogo-recursos.md` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `docs/ADR-007-catalogo-recursos.md` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.2.21 — Comparar todos los docs recibidos

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.2-recursos-verbos-codigos/AceriaApp.sln` y verificar que se está trabajando en el punto 2.2. **Acción 2:** Localizar `docs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.1-diseno-apis-rest. **Acción 3:** Comparar todos los docs recibidos. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `docs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `docs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Verificación íntegra en CMD desde la raíz del repositorio:

```
for %f in (M02_APIS_REST_ASPNET_CORE\PROYECTO\2.1-diseno-apis-rest\docs\*.md) do fc /L "%f"
"M02_APIS_REST_ASPNET_CORE\PROYECTO\2.2-recursos-verbos-codigos\docs\%~nxf"
```

Para un archivo .bat debe utilizarse %%f; desde CMD interactivo se usa %f.

PASO 2.2.22 — Examinar árbol de proyecto

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.2-recursos-verbos-codigos/AceriaApp.sln` y verificar que se está trabajando en el punto 2.2. **Acción 2:** Localizar `AceriaApp.sln` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.1-diseno-apis-rest. **Acción 3:** Examinar árbol de proyecto. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `AceriaApp.sln` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `AceriaApp.sln` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.2.23 — Comprobar .gitignore

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.2-recursos-verbos-codigos/AceriaApp.sln` y verificar que se está trabajando en el punto 2.2. **Acción 2:** Localizar `.gitignore` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.1-diseno-apis-rest. **Acción 3:** Comprobar .gitignore. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `.gitignore` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `.gitignore` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.2.24 — Confirmar el estado final

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.2-recursos-verbos-codigos/AceriaApp.sln` y verificar que se está trabajando en el punto 2.2. **Acción 2:** Localizar `AceriaApp.sln` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.1-diseno-apis-rest. **Acción 3:** Confirmar el estado final. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `AceriaApp.sln` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `AceriaApp.sln` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Tabla de errores del ejercicio completo

Error	Causa	Solución
No se encuentra la solución	Directorio incorrecto	Cambiar a la carpeta del punto y abrir AceriaApp.sln.
No se resuelve un tipo	Proyecto o dependencia ausente	Comprobar csproj y ejecutar dotnet restore.
No se publica Swagger	Entorno o middleware incorrecto	Contrastar Development con Production y consultar el JSON real.
Fallan tests anteriores	Cambio en el estado heredado	Restaurar el archivo alterado desde la carpeta anterior.
La documentación difiere del código	Contrato no actualizado	Comprobar ruta, modelo y respuesta con la API ejecutada.

Reto resuelto paso a paso

Reto: ampliar CatalogoCombinaciones con un resumen de válidas, inválidas y porcentaje.

Paso 1: Crear un record con Validas, Invalidas y PorcentajeValidez. **Paso 2:** Agregar el método Resumen dentro de la clase indicada. **Paso 3:** Contar las entradas válidas a través del validador. **Paso 4:** Calcular las inválidas como total menos válidas. **Paso 5:** Proteger el denominador cuando el total sea cero. **Paso 6:** Incorporar un test con tres válidas y dos inválidas. **Paso 7:** Ejecutar el proyecto de pruebas y comprobar 60%. **Paso 8:** Conservar la solución previa sin alteraciones.

Referencia de cálculo: `_combinaciones.Count(c => validador.Validar(c).EsValida)`. Esta ampliación está indicada en el código como reto, pero no forma parte del recuento base hasta que se implemente.

Resultado esperado

- Solución independiente con 8 proyectos; una API y 7 proyectos de tests.
- Recuento acumulado previsto de pruebas: 36.
- Todos los documentos y proyectos de los puntos anteriores permanecen accesibles.

Conclusión y enlace tras el punto 2.2

El punto 2.2 vincula recursos, verbos http y códigos de respuesta con clases y comprobaciones concretas. El alumnado puede inspeccionar el proyecto y explicar los resultados observados antes de abordar el siguiente punto.

Punto 2.3 — Validación de peticiones

Solución: `PROYECTO/2.3-validacion-peticiones/AceriaApp.sln`. **Origen:** 2.2-recursos-verbos-codigos.

PASO 2.3.1 — Copiar solución del punto 2.2

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.3-validacion-peticiones/AceriaApp.sln` y verificar que se está trabajando en el punto 2.3. **Acción 2:** Localizar `AceriaApp.sln` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.2-recursos-verbos-codigos. **Acción 3:** Copiar solución del punto 2.2. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `AceriaApp.sln` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `AceriaApp.sln` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.3.2 — Abrir solución 2.3

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.3-validacion-peticiones/AceriaApp.sln` y verificar que se está trabajando en el punto 2.3. **Acción 2:** Localizar `AceriaApp.sln` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.2-recursos-verbos-codigos. **Acción 3:** Abrir solución 2.3. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `AceriaApp.sln` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `AceriaApp.sln` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Consola de comprobación:

```
cd M02_APIS_REST_ASPNET_CORE/PROYECTO/2.3-validacion-peticiones
dotnet restore AceriaApp.sln
dotnet build AceriaApp.sln
```

PASO 2.3.3 — Comprobar documentos heredados

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.3-validacion-peticiones/AceriaApp.sln` y verificar que se está trabajando en el punto 2.3. **Acción 2:** Localizar `docs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.2-recursos-verbos-codigos. **Acción 3:** Comprobar documentos heredados. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `docs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `docs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.3.4 — Redactar contrato de validación

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.3-validacion-peticiones/AceriaApp.sln` y verificar que se está trabajando en el punto 2.3. **Acción 2:** Localizar `docs/validacion-peticiones.md` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.2-recursos-verbos-codigos. **Acción 3:** Redactar contrato de validación. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `docs/validacion-peticiones.md` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `docs/validacion-peticiones.md` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.3.5 — Incorporar paquete FluentValidation DI

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.3-validacion-peticiones/AceriaApp.sln` y verificar que se está trabajando en el punto 2.3. **Acción 2:** Localizar `src/AceriaApp.Monolito.Api/AceriaApp.Monolito.Api.csproj` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.2-recursos-verbos-codigos. **Acción 3:** Incorporar paquete FluentValidation DI. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `src/AceriaApp.Monolito.Api/AceriaApp.Monolito.Api.csproj` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `src/AceriaApp.Monolito.Api/AceriaApp.Monolito.Api.csproj` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.3.6 — Crear carpeta Validacion

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.3-validacion-peticiones/AceriaApp.sln` y verificar que se está trabajando en el punto 2.3. **Acción 2:** Localizar `src/AceriaApp.Monolito.Api/Validacion` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.2-recursos-verbos-codigos. **Acción 3:** Crear carpeta Validacion. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `src/AceriaApp.Monolito.Api/Validacion` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `src/AceriaApp.Monolito.Api/Validacion` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.3.7 — Crear CrearOrdenRequest

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.3-validacion-peticiones/AceriaApp.sln` y verificar que se está trabajando en el punto 2.3. **Acción 2:** Localizar `src/AceriaApp.Monolito.Api/Validacion/CrearOrdenRequest.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.2-recursos-verbos-codigos. **Acción 3:** Crear CrearOrdenRequest. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `src/AceriaApp.Monolito.Api/Validacion/CrearOrdenRequest.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `src/AceriaApp.Monolito.Api/Validacion/CrearOrdenRequest.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Código real de referencia:

```
namespace AceriaApp.Monolito.Api.Validacion;
public record CrearOrdenRequest(string Cliente,string Material,int Cantidad);
```

El alumno identifica tipo, parámetros, valor devuelto y caso de error en el archivo anterior.

PASO 2.3.8 — Crear CrearOrdenRequestValidator

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.3-validacion-peticiones/AceriaApp.sln` y verificar que se está trabajando en el punto 2.3. **Acción 2:** Localizar `src/AceriaApp.Monolito.Api/Validacion/CrearOrdenRequestValidator.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.2-recursos-verbos-codigos. **Acción 3:** Crear CrearOrdenRequestValidator. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `src/AceriaApp.Monolito.Api/Validacion/CrearOrdenRequestValidator.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `src/AceriaApp.Monolito.Api/Validacion/CrearOrdenRequestValidator.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Código real de referencia:

```
using FluentValidation;
namespace AceriaApp.Monolito.Api.Validacion;
public class CrearOrdenRequestValidator:AbstractValidator<CrearOrdenRequest>
{
    public CrearOrdenRequestValidator()
    {
        RuleFor(x=>x.Cliente).NotEmpty().WithMessage("El cliente es obligatorio.")
            .MaximumLength(100).WithMessage("El cliente no puede superar 100 caracteres.");
        RuleFor(x=>x.Material).NotEmpty().WithMessage("El material es obligatorio.")
            .MaximumLength(100).WithMessage("El material no puede superar 100 caracteres.");
        RuleFor(x=>x.Cantidad).GreaterThan(0).WithMessage("La cantidad debe ser mayor que cero.")
            .LessThanOrEqualTo(10000).WithMessage("La cantidad no puede superar 10000.");
    }
}
```

El alumno identifica tipo, parámetros, valor devuelto y caso de error en el archivo anterior.

PASO 2.3.9 — Crear ValidationFilter

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.3-validacion-peticiones/AceriaApp.sln` y verificar que se está trabajando en el punto 2.3. **Acción 2:** Localizar `src/AceriaApp.Monolito.Api/Validacion/ValidationFilter.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.2-recursos-verbos-codigos. **Acción 3:** Crear ValidationFilter. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `src/AceriaApp.Monolito.Api/Validacion/ValidationFilter.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `src/AceriaApp.Monolito.Api/Validacion/ValidationFilter.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Código real de referencia:

```
using FluentValidation;
namespace AceriaApp.Monolito.Api.Validacion;
public class ValidationFilter<T>:IEndpointFilter where T:class
{
    private readonly IValidator<T> _validator;
    public ValidationFilter(IValidator<T> validator) => _validator=validator;
    public async ValueTask<object?> InvokeAsync(EndpointFilterInvocationContext
context,EndpointFilterDelegate next)
    {
        var request=context.Arguments.OfType<T>().FirstOrDefault();
        if(request is null) return Results.BadRequest();
        var result=await _validator.ValidateAsync(request);
        if(!result.IsValid)
        {
            var errors=result.Errors.GroupBy(e=>e.PropertyName)
                .ToDictionary(g=>g.Key,g=>g.Select(e=>e.ErrorMessage).ToArray());
            return Results.ValidationProblem(errors);
        }
        return await next(context);
    }
}
```

El alumno identifica tipo, parámetros, valor devuelto y caso de error en el archivo anterior.

PASO 2.3.10 — Revisar errores agrupados

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.3-validacion-peticiones/AceriaApp.sln` y verificar que se está trabajando en el punto 2.3. **Acción 2:** Localizar `src/AceriaApp.Monolito.Api/Validacion/ValidationFilter.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.2-recursos-verbos-codigos. **Acción 3:** Revisar errores agrupados. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `src/AceriaApp.Monolito.Api/Validacion/ValidationFilter.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `src/AceriaApp.Monolito.Api/Validacion/ValidationFilter.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.3.11 — Añadir proyecto de pruebas

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.3-validacion-peticiones/AceriaApp.sln` y verificar que se está trabajando en el punto 2.3. **Acción 2:** Localizar `tests/AceriaApp.Validacion.Tests` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.2-recursos-verbos-codigos. **Acción 3:** Añadir proyecto de pruebas. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet test AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `tests/AceriaApp.Validacion.Tests` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `tests/AceriaApp.Validacion.Tests` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.3.12 — Añadir referencia al monolito

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.3-validacion-peticiones/AceriaApp.sln` y verificar que se está trabajando en el punto 2.3. **Acción 2:** Localizar `tests/AceriaApp.Validacion.Tests/AceriaApp.Validacion.Tests.csproj` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.2-recursos-verbos-codigos. **Acción 3:** Añadir referencia al monolito. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `tests/AceriaApp.Validacion.Tests/AceriaApp.Validacion.Tests.csproj` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `tests/AceriaApp.Validacion.Tests/AceriaApp.Validacion.Tests.csproj` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.3.13 — Comprobar xUnit y aserciones

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.3-validacion-peticiones/AceriaApp.sln` y verificar que se está trabajando en el punto 2.3. **Acción 2:** Localizar `tests/AceriaApp.Validacion.Tests/AceriaApp.Validacion.Tests.csproj` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.2-recursos-verbos-codigos. **Acción 3:** Comprobar xUnit y aserciones. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `tests/AceriaApp.Validacion.Tests/AceriaApp.Validacion.Tests.csproj` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `tests/AceriaApp.Validacion.Tests/AceriaApp.Validacion.Tests.csproj` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.3.14 — Probar reglas del validador

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.3-validacion-peticiones/AceriaApp.sln` y verificar que se está trabajando en el punto 2.3. **Acción 2:** Localizar `tests/AceriaApp.Validacion.Tests/CrearOrdenRequestValidatorTests.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.2-recursos-verbos-codigos. **Acción 3:** Probar reglas del validador. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `tests/AceriaApp.Validacion.Tests/CrearOrdenRequestValidatorTests.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `tests/AceriaApp.Validacion.Tests/CrearOrdenRequestValidatorTests.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.3.15 — Crear CrearOrdenRequestTests

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.3-validacion-peticiones/AceriaApp.sln` y verificar que se está trabajando en el punto 2.3. **Acción 2:** Localizar `tests/AceriaApp.Validacion.Tests/CrearOrdenRequestTests.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.2-recursos-verbos-codigos. **Acción 3:** Crear CrearOrdenRequestTests. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet test AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `tests/AceriaApp.Validacion.Tests/CrearOrdenRequestTests.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `tests/AceriaApp.Validacion.Tests/CrearOrdenRequestTests.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Código real de referencia:

```
using AceriaApp.Monolito.Api.Validacion;
using Xunit;
namespace AceriaApp.Validacion.Tests;
public class CrearOrdenRequestTests
{
    [Fact] public void Contrato_AsignaValores()
    {
        var p=new CrearOrdenRequest("ACX-001","CHAPA-3MM",10);
        Assert.Equal("ACX-001",p.Cliente); Assert.Equal("CHAPA-3MM",p.Material);
Assert.Equal(10,p.Cantidad);
    }
    [Fact] public void Contrato_PermiteRepresentarCadenaVacia()
    {
        var p=new CrearOrdenRequest("","CHAPA-3MM",10);
        Assert.Equal("",p.Cliente);
    }
}
```

El alumno identifica tipo, parámetros, valor devuelto y caso de error en el archivo anterior.

PASO 2.3.16 — Crear ValidacionExtensions e integrar Program

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.3-validacion-peticiones/AceriaApp.sln` y verificar que se está trabajando en el punto 2.3. **Acción 2:** Localizar `src/AceriaApp.Monolito.Api/Validacion/ValidacionExtensions.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.2-recursos-verbos-codigos. **Acción 3:** Crear ValidacionExtensions e integrar

Program. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `src/AceriaApp.Monolito.Api/Validacion/ValidacionExtensions.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `src/AceriaApp.Monolito.Api/Validacion/ValidacionExtensions.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Código real de referencia:

```
using FluentValidation;
namespace AceriaApp.Monolito.Api.Validacion;
public static class ValidacionExtensions
{
    public static IServiceCollection AddValidacion(this IServiceCollection services)
    {
        services.AddValidatorsFromAssemblyContaining<CrearOrdenRequestValidator>();
        return services;
    }
}
```

El alumno identifica tipo, parámetros, valor devuelto y caso de error en el archivo anterior.

PASO 2.3.17 — Compilar solución

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.3-validacion-peticiones/AceriaApp.sln` y verificar que se está trabajando en el punto 2.3. **Acción 2:** Localizar `AceriaApp.sln` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.2-recursos-verbos-codigos. **Acción 3:** Compilar solución. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `AceriaApp.sln` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `AceriaApp.sln` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.3.18 — Ejecutar seis tests nuevos

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.3-validacion-peticiones/AceriaApp.sln` y verificar que se está trabajando en el punto 2.3. **Acción 2:** Localizar `tests/AceriaApp.Validacion.Tests` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.2-recursos-verbos-codigos. **Acción 3:** Ejecutar seis tests nuevos. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet test AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `tests/AceriaApp.Validacion.Tests` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `tests/AceriaApp.Validacion.Tests` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.3.19 — Ejecutar pruebas acumuladas

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.3-validacion-peticiones/AceriaApp.sln` y verificar que se está trabajando en el punto 2.3. **Acción 2:** Localizar `AceriaApp.sln` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.2-recursos-verbos-codigos. **Acción 3:** Ejecutar pruebas acumuladas. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet test AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `AceriaApp.sln` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `AceriaApp.sln` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.3.20 — Explicar cambio aditivo en README

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.3-validacion-peticiones/AceriaApp.sln` y verificar que se está trabajando en el punto 2.3. **Acción 2:** Localizar `README.md` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.2-recursos-verbos-codigos. **Acción 3:** Explicar cambio aditivo en README. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `README.md` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `README.md` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.3.21 — Crear ADR-008

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.3-validacion-peticiones/AceriaApp.sln` y verificar que se está trabajando en el punto 2.3. **Acción 2:** Localizar `docs/ADR-008-validacion-peticiones.md` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.2-recursos-verbos-codigos. **Acción 3:** Crear ADR-008. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `docs/ADR-008-validacion-peticiones.md` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `docs/ADR-008-validacion-peticiones.md` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.3.22 — Comparar todos los docs previos

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.3-validacion-peticiones/AceriaApp.sln` y verificar que se está trabajando en el punto 2.3. **Acción 2:** Localizar `docs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.2-recursos-verbos-codigos. **Acción 3:**

Comparar todos los docs previos. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `docs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `docs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Verificación íntegra en CMD desde la raíz del repositorio:

```
for %f in (M02_APIS_REST_ASPNET_CORE\PROYECTO\2.2-recursos-verbos-codigos\docs\*.md) do fc /L "%f"
"M02_APIS_REST_ASPNET_CORE\PROYECTO\2.3-validacion-peticiones\docs\%~nxf"
```

Para un archivo .bat debe utilizarse %%f; desde CMD interactivo se usa %f.

PASO 2.3.23 — Verificar estructura y dependencias

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.3-validacion-peticiones/AceriaApp.sln` y verificar que se está trabajando en el punto 2.3. **Acción 2:** Localizar `src/AceriaApp.Monolito.Api/AceriaApp.Monolito.Api.csproj` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.2-recursos-verbos-codigos. **Acción 3:** Verificar estructura y dependencias. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `src/AceriaApp.Monolito.Api/AceriaApp.Monolito.Api.csproj` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `src/AceriaApp.Monolito.Api/AceriaApp.Monolito.Api.csproj` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.3.24 — Confirmar el estado final

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.3-validacion-peticiones/AceriaApp.sln` y verificar que se está trabajando en el punto 2.3. **Acción 2:** Localizar `AceriaApp.sln` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.2-recursos-verbos-codigos. **Acción 3:** Confirmar el estado final. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `AceriaApp.sln` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `AceriaApp.sln` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Tabla de errores del ejercicio completo

Error	Causa	Solución
No se encuentra la solución	Directorio incorrecto	Cambiar a la carpeta del punto y abrir AceriaApp.sln.
No se resuelve un tipo	Proyecto o dependencia ausente	Comprobar csproj y ejecutar dotnet restore.
No se publica Swagger	Entorno o middleware incorrecto	Contrastar Development con Production y consultar el JSON real.
Fallan tests anteriores	Cambio en el estado heredado	Restaurar el archivo alterado desde la carpeta anterior.
La documentación difiere del código	Contrato no actualizado	Comprobar ruta, modelo y respuesta con la API ejecutada.

Reto resuelto paso a paso

Reto: crear ActualizarOrdenRequest y un validador que rechace Guid.Empty.

Paso 1: Crear el record con Guid Id, Cliente, Material y Cantidad. **Paso 2:** Crear AbstractValidator del nuevo record. **Paso 3:** Configurar RuleFor(x => x.Id).NotEmpty(). **Paso 4:** Añadir reglas de límites de cantidad. **Paso 5:** Crear un test con Guid.Empty. **Paso 6:** Crear un test con Guid.NewGuid(). **Paso 7:** Ejecutar dotnet test y comprobar ocho tests en el proyecto. **Paso 8:** Verificar que no se modificó 2.2.

Resultado esperado

- Solución independiente con 9 proyectos; una API y 8 proyectos de tests.

- Recuento acumulado previsto de pruebas: 42.
- Todos los documentos y proyectos de los puntos anteriores permanecen accesibles.

Conclusión y enlace tras el punto 2.3

El punto 2.3 vincula validación de peticiones con clases y comprobaciones concretas. El alumnado puede inspeccionar el proyecto y explicar los resultados observados antes de abordar el siguiente punto.

Punto 2.4 — Documentación mediante OpenAPI (Swagger)

Solución: `PROYECTO/2.4-documentacion-openapi/AceriaApp.sln`. **Origen:** 2.3-validacion-peticiones.

PASO 2.4.1 — Copiar solución del punto 2.3

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.4-documentacion-openapi/AceriaApp.sln` y verificar que se está trabajando en el punto 2.4. **Acción 2:** Localizar `AceriaApp.sln` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.3-validacion-peticiones. **Acción 3:** Copiar solución del punto 2.3. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `AceriaApp.sln` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `AceriaApp.sln` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.4.2 — Abrir solución 2.4

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.4-documentacion-openapi/AceriaApp.sln` y verificar que se está trabajando en el punto 2.4. **Acción 2:** Localizar `AceriaApp.sln` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.3-validacion-peticiones. **Acción 3:** Abrir solución 2.4. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `AceriaApp.sln` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `AceriaApp.sln` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Consola de comprobación:

```
cd M02_APIS_REST_ASPNET_CORE/PROYECTO/2.4-documentacion-openapi
dotnet restore AceriaApp.sln
dotnet build AceriaApp.sln
```

PASO 2.4.3 — Revisar documentos heredados

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.4-documentacion-openapi/AceriaApp.sln` y verificar que se está trabajando en el punto 2.4. **Acción 2:** Localizar `docs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.3-validacion-peticiones. **Acción 3:** Revisar documentos heredados. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `docs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `docs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.4.4 — Crear guía de documentación

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.4-documentacion-openapi/AceriaApp.sln` y verificar que se está trabajando en el punto 2.4. **Acción 2:** Localizar `docs/documentacion-openapi.md` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.3-validacion-peticiones. **Acción 3:** Crear guía de documentación. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `docs/documentacion-openapi.md` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `docs/documentacion-openapi.md` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.4.5 — Crear carpeta Documentacion

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.4-documentacion-openapi/AceriaApp.sln` y verificar que se está trabajando en el punto 2.4. **Acción 2:** Localizar `src/AceriaApp.Monolito.Api/Documentacion` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.3-validacion-peticiones. **Acción 3:** Crear carpeta Documentacion. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `src/AceriaApp.Monolito.Api/Documentacion` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `src/AceriaApp.Monolito.Api/Documentacion` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.4.6 — Crear MetadatosApi

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.4-documentacion-openapi/AceriaApp.sln` y verificar que se está trabajando en el punto 2.4. **Acción 2:** Localizar `src/AceriaApp.Monolito.Api/Documentacion/MetadatosApi.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.3-validacion-peticiones. **Acción 3:** Crear MetadatosApi. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `src/AceriaApp.Monolito.Api/Documentacion/MetadatosApi.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.

- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `src/AceriaApp.Monolito.Api/Documentacion/MetadatosApi.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Código real de referencia:

```
namespace AceriaApp.Monolito.Api.Documentacion;
public class MetadatosApi
{
    public string Titulo {get;init;}="AceriaApp.Monolito.Api";
    public string Version {get;init;}="v1";
    public string Descripcion {get;init;}="API del monolito de referencia.";
    public string ContactoNombre {get;init;}="Equipo AceriaApp";
    public string ContactoEmail {get;init;}="equipo@aceria.local";
}
```

El alumno identifica tipo, parámetros, valor devuelto y caso de error en el archivo anterior.

PASO 2.4.7 — Crear ConfiguracionOpenApi

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.4-documentacion-openapi/AceriaApp.sln` y verificar que se está trabajando en el punto 2.4. **Acción 2:** Localizar `src/AceriaApp.Monolito.Api/Documentacion/ConfiguracionOpenApi.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.3-validacion-peticiones. **Acción 3:** Crear ConfiguracionOpenApi. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `src/AceriaApp.Monolito.Api/Documentacion/ConfiguracionOpenApi.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `src/AceriaApp.Monolito.Api/Documentacion/ConfiguracionOpenApi.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Código real de referencia:

```
namespace AceriaApp.Monolito.Api.Documentacion;
public class ConfiguracionOpenApi
{
    public MetadatosApi Metadatos {get;init;}=new();
    public bool PublicarSwaggerUI {get;init;}=true;
    public string RutaDocumento {get;init;}="/swagger/v1/swagger.json";
    public string NombreDocumento {get;init;}="AceriaApp.Monolito.Api v1";
    public bool EsEntornoDeDesarrollo(string entorno) =>
        string.Equals(entorno,"Development",StringComparison.OrdinalIgnoreCase);
    // RETO 2.4: añadir EsEntornoDeProduccion y su test.
}
```

El alumno identifica tipo, parámetros, valor devuelto y caso de error en el archivo anterior.

PASO 2.4.8 — Comprobar selección del entorno

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.4-documentacion-openapi/AceriaApp.sln` y verificar que se está trabajando en el punto 2.4. **Acción 2:** Localizar `src/AceriaApp.Monolito.Api/Documentacion/ConfiguracionOpenApi.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.3-validacion-peticiones. **Acción 3:** Comprobar selección del entorno. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `src/AceriaApp.Monolito.Api/Documentacion/ConfiguracionOpenApi.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `src/AceriaApp.Monolito.Api/Documentacion/ConfiguracionOpenApi.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Código real de referencia:

```
namespace AceriaApp.Monolito.Api.Documentacion;
public class ConfiguracionOpenApi
{
    public MetadatosApi Metadatos {get;init;}=new();
    public bool PublicarSwaggerUI {get;init;}=true;
    public string RutaDocumento {get;init;}="/swagger/v1/swagger.json";
    public string NombreDocumento {get;init;}="AceriaApp.Monolito.Api v1";
    public bool EsEntornoDeDesarrollo(string entorno) =>
        string.Equals(entorno,"Development",StringComparison.OrdinalIgnoreCase);
    // RETO 2.4: añadir EsEntornoDeProduccion y su test.
}
```

El alumno identifica tipo, parámetros, valor devuelto y caso de error en el archivo anterior.

PASO 2.4.9 — Crear DocumentacionExtensions e integrar Swagger

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.4-documentacion-openapi/AceriaApp.sln` y verificar que se está trabajando en el punto 2.4. **Acción 2:** Localizar `src/AceriaApp.Monolito.Api/Documentacion/DocumentacionExtensions.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.3-validacion-peticiones. **Acción 3:** Crear DocumentacionExtensions e integrar Swagger. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `src/AceriaApp.Monolito.Api/Documentacion/DocumentacionExtensions.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `src/AceriaApp.Monolito.Api/Documentacion/DocumentacionExtensions.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Código real de referencia:

```
using Microsoft.OpenApi.Models;
namespace AceriaApp.Monolito.Api.Documentacion;
public static class DocumentacionExtensions
{
    public static IServiceCollection AddDocumentacionOpenApi(
        this IServiceCollection services,MetadatosApi metadatos)
    {
        services.AddEndpointsApiExplorer();
        services.AddSwaggerGen(options => options.SwaggerDoc(metadatos.Version,new OpenApiInfo
        {
            Title=metadatos.Titulo,Version=metadatos.Version,
            Description=metadatos.Descripcion,
            Contact=new OpenApiContact
{ Name=metadatos.ContactoNombre,Email=metadatos.ContactoEmail}
        }));
        return services;
    }
    public static WebApplication UseDocumentacionOpenApi(
        this WebApplication app,ConfiguracionOpenApi configuracion)
    {
        if(configuracion.EsEntornoDeDesarrollo(app.Environment.EnvironmentName))
        {
            app.UseSwagger();
            if(configuracion.PublicarSwaggerUI)
                app.UseSwaggerUI(options=>options.SwaggerEndpoint(
                    configuracion.RutaDocumento,configuracion.NombreDocumento));
        }
        return app;
    }
}
```

El alumno identifica tipo, parámetros, valor devuelto y caso de error en el archivo anterior.

PASO 2.4.10 — Añadir proyecto de pruebas

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.4-documentacion-openapi/AceriaApp.sln` y verificar que se está trabajando en el punto 2.4. **Acción 2:** Localizar `tests/AceriaApp.Documentacion.Tests` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.3-validacion-peticiones. **Acción 3:** Añadir proyecto de pruebas. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet test AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `tests/AceriaApp.Documentacion.Tests` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `tests/AceriaApp.Documentacion.Tests` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.4.11 — Añadir referencia al monolito

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.4-documentacion-openapi/AceriaApp.sln` y verificar que se está trabajando en el punto 2.4. **Acción 2:** Localizar `tests/AceriaApp.Documentacion.Tests/AceriaApp.Documentacion.Tests.csproj` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.3-validacion-peticiones. **Acción 3:** Añadir referencia al monolito. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `tests/AceriaApp.Documentacion.Tests/AceriaApp.Documentacion.Tests.csproj` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `tests/AceriaApp.Documentacion.Tests/AceriaApp.Documentacion.Tests.csproj` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.4.12 — Añadir Microsoft.AspNetCore.Mvc.Testing

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.4-documentacion-openapi/AceriaApp.sln` y verificar que se está trabajando en el punto 2.4. **Acción 2:** Localizar `tests/AceriaApp.Documentacion.Tests/AceriaApp.Documentacion.Tests.csproj` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.3-validacion-peticiones. **Acción 3:** Añadir Microsoft.AspNetCore.Mvc.Testing. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet test AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `tests/AceriaApp.Documentacion.Tests/AceriaApp.Documentacion.Tests.csproj` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `tests/AceriaApp.Documentacion.Tests/AceriaApp.Documentacion.Tests.csproj` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.4.13 — Probar datos de contacto

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.4-documentacion-openapi/AceriaApp.sln` y verificar que se está trabajando en el punto 2.4. **Acción 2:** Localizar `tests/AceriaApp.Documentacion.Tests/MetadatosApiTests.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.3-validacion-peticiones. **Acción 3:** Probar datos de contacto. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `tests/AceriaApp.Documentacion.Tests/MetadatosApiTests.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `tests/AceriaApp.Documentacion.Tests/MetadatosApiTests.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.4.14 — Probar configuración

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.4-documentacion-openapi/AceriaApp.sln` y verificar que se está trabajando en el punto 2.4. **Acción 2:** Localizar `tests/AceriaApp.Documentacion.Tests/ConfiguracionOpenApiTests.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.3-validacion-peticiones. **Acción 3:** Probar configuración. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `tests/AceriaApp.Documentacion.Tests/ConfiguracionOpenApiTests.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `tests/AceriaApp.Documentacion.Tests/ConfiguracionOpenApiTests.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.4.15 — Probar Swagger HTTP con WebApplicationFactory

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.4-documentacion-openapi/AceriaApp.sln` y verificar que se está trabajando en el punto 2.4. **Acción 2:** Localizar `tests/AceriaApp.Documentacion.Tests/DocumentacionExtensionsTests.cs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.3-validacion-peticiones. **Acción 3:** Probar Swagger HTTP con WebApplicationFactory. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `tests/AceriaApp.Documentacion.Tests/DocumentacionExtensionsTests.cs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `tests/AceriaApp.Documentacion.Tests/DocumentacionExtensionsTests.cs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Código real de referencia:

```
using Microsoft.AspNetCore.Mvc.Testing;
using Microsoft.AspNetCore.Hosting;
using System.Text.Json;
using System.Net.Http.Json;
using Xunit;
namespace AceriaApp.Documentacion.Tests;
public class DocumentacionExtensionsTests
{
    [Fact] public async Task Swagger_EnDesarrollo_DevuelveJsonOpenApi()
    {
        using var f=new
WebApplicationFactory<Program>().WithWebHostBuilder(b=>b.UseEnvironment("Development"));
        using var cliente=f.CreateClient();
        var respuesta=await cliente.GetAsync("/swagger/v1/swagger.json");
        respuesta.EnsureSuccessStatusCode();
        using var json=JsonDocument.Parse(await respuesta.Content.ReadAsStringAsync());
        Assert.True(json.RootElement.TryGetProperty("openapi",out _));
    }
    [Fact] public async Task Swagger_ContieneVersion()
    {
        using var f=new
WebApplicationFactory<Program>().WithWebHostBuilder(b=>b.UseEnvironment("Development"));
        using var json=JsonDocument.Parse(await f.CreateClient().GetStringAsync("/swagger/v1/
swagger.json"));
        Assert.Equal("v1",json.RootElement.GetProperty("info").GetProperty("version").GetString());
    }
    [Fact] public async Task Swagger_DocumentaOrdenes()
    {
        using var f=new
WebApplicationFactory<Program>().WithWebHostBuilder(b=>b.UseEnvironment("Development"));
        using var json=JsonDocument.Parse(await f.CreateClient().GetStringAsync("/swagger/v1/
swagger.json"));
        Assert.True(json.RootElement.GetProperty("paths").TryGetProperty("/api/v1/ordenes",out _));
        var invalid = await f.CreateClient().PostAsJsonAsync("/api/v1/ordenes", new { cliente="",
material="CHAPA-3MM", cantidad=-1 });
        Assert.Equal(System.Net.HttpStatusCode.BadRequest, invalid.StatusCode);
    }
    [Fact] public async Task Swagger_NoSePublicaEnProduccion()
    {
        using var f=new
WebApplicationFactory<Program>().WithWebHostBuilder(b=>b.UseEnvironment("Production"));
        var response=await f.CreateClient().GetAsync("/swagger/v1/swagger.json");
        Assert.Equal(System.Net.HttpStatusCode.NotFound,response.StatusCode);
    }
}
```

El alumno identifica tipo, parámetros, valor devuelto y caso de error en el archivo anterior.

PASO 2.4.16 — Compilar solución

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.4-documentacion-openapi/AceriaApp.sln` y verificar que se está trabajando en el punto 2.4. **Acción 2:** Localizar `AceriaApp.sln` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.3-validacion-peticiones. **Acción 3:** Compilar solución. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `AceriaApp.sln` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `AceriaApp.sln` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.4.17 — Ejecutar nueve tests nuevos

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.4-documentacion-openapi/AceriaApp.sln` y verificar que se está trabajando en el punto 2.4. **Acción 2:** Localizar `tests/AceriaApp.Documentacion.Tests` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.3-validacion-peticiones. **Acción 3:** Ejecutar nueve tests nuevos. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet test AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `tests/AceriaApp.Documentacion.Tests` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `tests/AceriaApp.Documentacion.Tests` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.4.18 — Ejecutar tests acumulados

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.4-documentacion-openapi/AceriaApp.sln` y verificar que se está trabajando en el punto 2.4. **Acción 2:** Localizar `AceriaApp.sln` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.3-validacion-peticiones. **Acción 3:** Ejecutar tests acumulados. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet test AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `AceriaApp.sln` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.

- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `AceriaApp.sln` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.4.19 — Actualizar README

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.4-documentacion-openapi/AceriaApp.sln` y verificar que se está trabajando en el punto 2.4. **Acción 2:** Localizar `README.md` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.3-validacion-peticiones. **Acción 3:** Actualizar README. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `README.md` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `README.md` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.4.20 — Crear ADR-009

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.4-documentacion-openapi/AceriaApp.sln` y verificar que se está trabajando en el punto 2.4. **Acción 2:** Localizar `docs/ADR-009-documentacion-openapi.md` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.3-validacion-peticiones. **Acción 3:** Crear ADR-009. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `docs/ADR-009-documentacion-openapi.md` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `docs/ADR-009-documentacion-openapi.md` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.4.21 — Comparar documentación previa

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.4-documentacion-openapi/AceriaApp.sln` y verificar que se está trabajando en el punto 2.4. **Acción 2:** Localizar `docs` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.3-validacion-peticiones. **Acción 3:** Comparar documentación previa. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `docs` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `docs` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Verificación íntegra en CMD desde la raíz del repositorio:

```
for %f in (M02_APIS_REST_ASPNET_CORE\PROYECTO\2.3-validacion-peticiones\docs\*.md) do fc /L "%f"
"M02_APIS_REST_ASPNET_CORE\PROYECTO\2.4-documentacion-openapi\docs\%~nxf"
```

Para un archivo .bat debe utilizarse %%f; desde CMD interactivo se usa %f.

PASO 2.4.22 — Comprobar árbol final

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.4-documentacion-openapi/AceriaApp.sln` y verificar que se está trabajando en el punto 2.4. **Acción 2:** Localizar `AceriaApp.sln` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.3-validacion-peticiones. **Acción 3:** Comprobar árbol final. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `AceriaApp.sln` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `AceriaApp.sln` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.4.23 — Revisar archivos excluidos

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.4-documentacion-openapi/AceriaApp.sln` y verificar que se está trabajando en el punto 2.4. **Acción 2:** Localizar `.gitignore` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.3-validacion-peticiones. **Acción 3:** Revisar archivos excluidos. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `.gitignore` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `.gitignore` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

PASO 2.4.24 — Cerrar módulo 2

Acción 1: Abrir en Visual Studio la solución `PROYECTO/2.4-documentacion-openapi/AceriaApp.sln` y verificar que se está trabajando en el punto 2.4. **Acción 2:** Localizar `AceriaApp.sln` y comprobar su función dentro del proyecto acumulativo. No modificar la solución correspondiente a 2.3-validacion-peticiones. **Acción 3:** Cerrar módulo 2. Revisar el contenido completo del elemento, las entradas necesarias y el resultado esperado, comparándolos con la explicación teórica del punto. **Acción 4:** Desde la carpeta de la solución ejecutar `dotnet build AceriaApp.sln` y comprobar la salida antes de continuar.

Por qué se hace: La actividad proporciona evidencia comprobable del elemento `AceriaApp.sln` y mantiene la trazabilidad desde la teoría hasta el proyecto reproducible. El alumno debe distinguir un archivo preparado de un comportamiento conectado al servidor.

Errores comunes:

- Editar el mismo archivo en la solución del punto previo y perder el estado acumulado.
- Suponer que un método de extensión se ejecuta solo por estar definido en una clase.
- Interpretar la presencia de un test como prueba de un comportamiento diferente del realmente comprobado.

Verificación: Confirmar que el archivo o carpeta `AceriaApp.sln` existe y que la compilación y las pruebas del punto terminan sin errores. Si falla la comprobación, identificar el nombre del proyecto y la aserción afectada.

Tabla de errores del ejercicio completo

Error	Causa	Solución
No se encuentra la solución	Directorio incorrecto	Cambiar a la carpeta del punto y abrir AceriaApp.sln.
No se resuelve un tipo	Proyecto o dependencia ausente	Comprobar csproj y ejecutar dotnet restore.
No se publica Swagger	Entorno o middleware incorrecto	Contrastar Development con Production y consultar el JSON real.
Fallan tests anteriores	Cambio en el estado heredado	Restaurar el archivo alterado desde la carpeta anterior.
La documentación difiere del código	Contrato no actualizado	Comprobar ruta, modelo y respuesta con la API ejecutada.

Reto resuelto paso a paso

Reto: añadir EsEntornoDeProduccion a ConfiguracionOpenApi.

Paso 1: Abrir ConfiguracionOpenApi.cs. **Paso 2:** Implementar string.Equals con Production y OrdinalIgnoreCase. **Paso 3:** Guardar sin cambiar EsEntornoDeDesarrollo. **Paso 4:** Incorporar test para Production. **Paso 5:** Incorporar aserción negativa para Development. **Paso 6:** Ejecutar los tests de Documentacion.Tests. **Paso 7:** Volver a comprobar la ausencia de Swagger JSON en Production. **Paso 8:** Conservar el punto 2.3 sin cambios.

Resultado esperado

- Solución independiente con 10 proyectos; una API y 9 proyectos de tests.
- Recuento acumulado previsto de pruebas: 51.
- Todos los documentos y proyectos de los puntos anteriores permanecen accesibles.

Conclusión y enlace tras el punto 2.4

El punto 2.4 vincula documentación mediante openapi (swagger) con clases y comprobaciones concretas. El alumnado puede inspeccionar el proyecto y explicar los resultados observados antes de abordar el siguiente punto.
